

Problem Sheet 2 - Intermediate SQL (With Solutions)

George Melrose: george_melrose@yahoo.com, <https://github.com/georgemelrose>

Introduction

These problems aim to test your intermediate SQL knowledge, building on the basic SQL concepts tested in problem sheet 1. The questions and solutions are of a more esoteric nature than problem sheet 1 yet still useful as a SQL coder. For the purposes of this series of problem sheets, a database of dummy Marathon results data has been generated. More information on the **Marathon** database is presented below.

The concepts tested in this sheet are covered by the LinkedIn learning course **Intermediate SQL for Data Scientists** - (<https://www.linkedin.com/learning/intermediate-sql-for-data-scientists/>) .

Useful Preparatory Resources

In addition to this problem sheet, there are two useful resources you can draw upon to better understand these SQL concepts:

- **Two RMarkdown documents** - one to generate some dummy ‘Universities’ data (https://github.com/georgemelrose/SQL_Practice/blob/main/0_generating_databasesstar_dummy_data.Rmd). This was copied from the excellent SQL learning resource databasesstar (https://github.com/bbrunm/databasesstar/tree/main/sample_databases/sample_db_university/sqlite).
The other document is an RMD HTML going over intermediate SQL concepts and how they can be applied to databasesstar dummy data (https://github.com/georgemelrose/SQL_Practice/blob/main/03_Intermediate_SQL_for_Data_Scientists.html).
- **A video presentation** - a recording of a meeting in which I presented the **Intermediate SQL for Data Scientists** HTML , explaining varying higher level concepts- (https://universityofcambridgecloud.sharepoint.com/sites/AD_Progress/SitePages/Learning-SQL-in-a-New-Format.aspx).

Marathon Database

Firstly, the data to be put into the Marathon database was formulated from the following Python script - (https://github.com/georgemelrose/SQL_Practice/blob/main/Dummy_Marathon_Data/marathon_data_generation.ipynb).

The **marathon data generation** python script generates the following tables:

1. **Runners** - Randomly generate 1000 runners with names common in their locale/country, together with their birth date and sex.
2. **Events** - The 6 Major World marathons (Berlin, Boston, Chicago,London,New York City, Tokyo), with an event per year from 2012 to 2023.
3. **Results** - Gives results for runners in hh:mm:ss format, ensuring there aren’t duplicate results for each runner per event. Prevents any results breaking either the male marathon world-record (2:00:35 Eliud

Kipchoge 2023) or the female marathon world-record (2:11:53 Brigid Kosgei 2019). Also determines, with a True/False column, if a result is elite by the male standard (below 02:15:00) or the female standard (below 02:30:00).

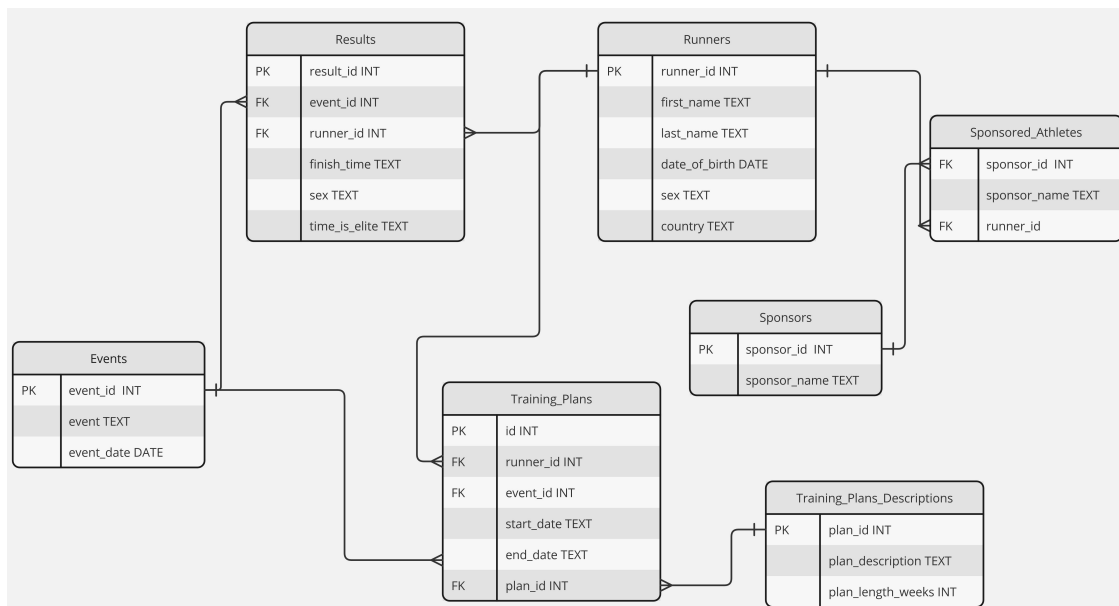
4. Sponsors - Lists the following 10 major companies that typically act as sponsors to runners - “Nike”, “Adidas”, “Asics”, “Saucony”, “Hoka”, “Brooks”, “New Balance”, “Puma”, “Under Armour”, “Tracksmith”.

5. Sponsored Athletes - A table listing the fraction of the elite athletes that have a sponsor.

6. Training Plans Descriptions - The descriptions of 10 different training plans and their respective lengths in weeks.

7. Training Plans - The training plans of athletes. Only 72% of runner-event combinations have an associated training plan.

Marathon Database Entity Relationship Diagram



Intermediate SQL Problems

Indexes

Q1. - Create a basic index on the `runner_id` column in the **Results** table to speed up searches and joins involving runner information?

Solution -

CREATE INDEX idx_results_runner_id ON Results(runner_id);

The **CREATE INDEX** command creates an index, given a name by the user, and is based off of a specified column(s).

Q2. - Construct a unique index on the `event_id` column of the **Events** table to ensure no duplicate event entries exist?

Solution -

CREATE UNIQUE INDEX idx_events_event_id ON Events(event_id);

A unique index is created by using the **CREATE UNIQUE INDEX** command on the `event_id` column from Events - the primary key for that table.

Q3. - Create a composite index on the `runner_id` and `event_id` columns in the Training_Plans table to improve performance for queries that frequently search for a specific runner's training plan for an event?

Solution -

```
CREATE INDEX idx_training_plans_runner_event ON Training_Plans(runner_id, event_id);
```

To create a *composite index*, the columns from the relevant table are selected in the **CREATE INDEX** command.

Q4. - Add a unique index on the `first_name`, `last_name`, and `date_of_birth` columns in the Runners table to ensure no duplicate runner profiles are entered, even if runners have similar names?

Solution -

```
CREATE UNIQUE INDEX idx_runners_name_dob ON Runners(first_name, last_name, date_of_birth);
```

To prevent duplicate entries in the Runners table, a *unique index* is created on the `first_name`, `last_name`, and `date_of_birth` columns using the **CREATE UNIQUE INDEX** command. This ensures that no two entries in the table can have the same values for all three columns, even if some names are similar.

Q5. - Give the code to see all the indexes present?

Solution -

```
SELECT name FROM sqlite_master WHERE type = 'index';
```

To view all indexes in the SQLite database, a query is made to the `sqlite_master` table. This table contains schema information, and by selecting only rows where type is 'index', we can list all indexes currently present.

Q6. - Give the command to see all the indexes associated with a particular table, for example 'Runners'?

Solution -

```
PRAGMA index_list('Runners');
```

To view all indexes for a specific table in SQLite, the **PRAGMA index_list** command is used. This command shows details of all indexes associated with the specified table, in this case, the Runners table.

Views

Q1. - Create a view of all male runners from the Baltic States? Also write code to display all the records from this view? **Hint** - most of the code is already present in problem sheet 1. Write code to delete a view if necessary (needed if you make a mistake in your code but the view is nevertheless created)

Solution -

```
CREATE VIEW Baltic_Men AS SELECT * FROM Runners WHERE country in ('Lithuania', 'Latvia', 'Estonia') AND sex = 'Male';
```

```
SELECT * FROM Baltic_Men;
```

```
DROP VIEW IF EXISTS table_name;
```

The code in the first part of this three-pronged solution is present as a solution on page 3 of Problem Sheet 1 albeit for this question, the additional condition of `sex = 'Male'` is necessary.

Now the view 'Baltic Men' can be queried like a regular table, using the **SELECT** command. The last command, **DROP VIEW**, deletes the view if needed.

Q2. - Create a view and display its' records, of all the Nike sponsored runners?

Solution -

```
CREATE VIEW Nike_Runners AS SELECT r.first_name || ' ' || r.last_name AS full_name, r.country, r.sex FROM Runners r LEFT OUTER JOIN Sponsored_Athletes sa on r.runner_id = sa.runner_id WHERE sponsor_name = 'Nike';
```

```
SELECT * FROM Nike_Runners;
```

To create a view of Nike-sponsored runners, use CREATE VIEW with a SELECT query that joins Runners and Sponsored_Athletes on runner_id, filtering where sponsor_name is 'Nike'.

Q3. - Create a view and display its' records, of the full names, finish times, and countries of UK runners who had an elite time in the Berlin Marathon (any year)? View all records from this view and write a query to highlight just the middle 5 rows, having arranged them from fastest to slowest time?

Solution -

```
CREATE VIEW Elite_Berlin_Brits AS SELECT r.first_name || ' ' || r.last_name AS full_name, r.country, re.finish_time FROM Runners INNER JOIN Results re ON re.runner_id = r.runner_id INNER JOIN Events e ON e.event_id = re.event_id WHERE event = 'Berlin Marathon' AND time_is_elite = 'True' AND country = 'United Kingdom';
```

To create a view of UK runners who had elite times in the Berlin Marathon, use CREATE VIEW with a SELECT query that joins Runners, Results, and Events on runner_id and event_id, filtering by event name, time_is_elite, and country.

```
SELECT * FROM Elite_Berlin_Brits;
```

First, determine the total no.rows in the view -

```
SELECT COUNT(*) AS total_rows FROM Elite_Berlin_Brits;
```

Then as there are 15 rows, display only the middle 5 in the results query by using **LIMIT 5 OFFSET 5**.

```
SELECT * FROM Elite_Berlin_Brits ORDER BY finish_time ASC LIMIT 5 OFFSET 5;
```

Q4. - Generate SQL code to “drop” a view, see all the views present, and see the SQL code from the view from question 3?

To drop a view, see all views, and retrieve the SQL code for a specific view, use the following commands:

```
DROP VIEW IF EXISTS Nike_Runners;
```

```
SELECT name FROM sqlite_master WHERE type='view';
```

```
SELECT sql FROM sqlite_master WHERE type='view' AND name = 'Elite_Berlin_Brits';
```

Statistical Aggregate Functions

Q1. - Count the number of distinct runners and runners' countries from the 'Runners' table?

```
SELECT COUNT(DISTINCT runner_id) FROM Runners;
```

```
SELECT COUNT(DISTINCT country) FROM Runners;
```

Using the COUNT() function in conjunction with the DISTINCT() allows the number of distinct entries in either column to be found. Be careful not to try to arbitrarily use SUM() when finding unique entries in a column - it only works on integer variables!

Q2. - For those runners with a training plan, find out how many weeks they cumulatively trained across all their training plans? Order this by the most weeks trained in total to the least?

```
SELECT r.first_name || ' ' || r.last_name AS full_name, SUM(tpd.plan_length_weeks)
FROM Runners r INNER JOIN Training_Plans tp ON r.runner_id = tp.runner_id IN-
NER JOIN Training_Plans_Descriptions tpd ON tp.plan_id = tpd.plan_id GROUP BY
r.runner_id, r.first_name, r.last_name ORDER BY SUM(tpd.plan_length_weeks) DESC;
```

To calculate the total weeks trained for each runner with a training plan, use the **SUM** function to add up the `plan_length_weeks` values for each runner. The **INNER JOIN** ensures that only runners with training plans are included, and the **GROUP BY** groups by each runner's ID and name to calculate a total per runner. The result is ordered from the most to the least total weeks trained.

Q3. - For runners with more than one training plan, generate a query result that shows their average training plan length?

```
SELECT r.first_name || ' ' || r.last_name AS full_name, AVG(tpd.plan_length_weeks)
FROM Runners r INNER JOIN Training_Plans tp ON r.runner_id = tp.runner_id IN-
NER JOIN Training_Plans_Descriptions tpd ON tp.plan_id = tpd.plan_id GROUP BY
r.runner_id, r.first_name, r.last_name HAVING COUNT(tp.plan_id) > 1;
```

To calculate the average training plan length for runners with more than one training plan, the **AVG** function is used on `plan_length_weeks`. The **INNER JOIN** operations ensure only runners with training plans are included. After grouping by each runner's ID and name, only those runners with multiple training plans are displayed by adding a **HAVING** clause.

Q4. - For the query result in the previous question, round the number given for the average training plan length to the nearest week?

```
SELECT r.first_name || ' ' || r.last_name AS full_name, ROUND(AVG(tpd.plan_length_weeks))
FROM Runners r INNER JOIN Training_Plans tp ON r.runner_id = tp.runner_id IN-
NER JOIN Training_Plans_Descriptions tpd ON tp.plan_id = tpd.plan_id GROUP BY
r.runner_id, r.first_name, r.last_name HAVING COUNT(tp.plan_id) > 1;
```

To round the average training plan length to the nearest week, the **ROUND** function is used around the **AVG** function. This rounds the average value to the nearest whole number.

WITH() Function / Common Table Expressions